Sets

Sets are used to store multiple items in a single variable.A set is created by placing all the items (elements) inside curly braces {}, separated by comma.

```
set1={'aa','bb','cc','dd'}
set2=set(('aaa','bbb','ccc','ddd'))
print(set1)
print(set2)
```

Sets are unordered ,unchangeable and unindexed.

Note: Set items are unchangeable, but items can be removed and new items can be added.

Sets can have any number of items and they may be of different types (integer, float, tuple, string etc.). But a set cannot have mutable elements like lists, sets or dictionaries as its elements.

```
lang={'C','C++','Java','Python','Perl'}
print(lang)
s1={1,2,3,4}
print(s1)
s2={2.2,3.4,4.5}
print(s2)
s3={1,2.3,'abc'}
print(s3)
s4={True,False,False}
print(s4)
```

Sets cannot have duplicate elements.If duplicate elements are present then they are ignored.

```
lang={'C','C++','Java','Python','Perl','Java'}
print(lang)
```

Datatype of set:

The datatype of set can be know with type().

```
lang={'C','C++','Java','Python','Perl'}
print(type(lang))
```

Accessing elements of set

```
for i in lang:
  print(i)
```

Operations on Sets

Computing length of set

length of set can be computed using len().

```
lang={'C','C++','Java','Python','Perl'}
print(len(lang))
```

Membership operators (in and not in)

in operator returns True returns if the element is present otherwise returns False. not in operator returns True returns if the element is not present otherwise returns False.

```
lang={'C','C++','Java','Python','Perl'}
print('C++' in lang)
print('Ruby' in lang)
print('C' not in lang)
print('C#' not in lang)
```

other operations

max(): returns maximum element from a set
min(): returns minimum element from a set
sum(): returns the sum of elements from a set
sorted(): return a new sorted list from elements in the list.
copy(): Returns copy of a list.

```
>>>s1={1,2,3,4,5}
>>> s3=s1.copy()
>>> s3
{1, 2, 3, 4, 5}
```

Set Methods

Once a set is created, you cannot change its items, but you can add new items. To add one item to a set use the add() method.
To add items from another set into the current set, use the update() method.

```
>>> s={1,2,3.5,'psp'}
>>> s.add(90)
>>> s
{1, 2, 3.5, 'psp', 90}
>>> s.update({12,45})
>>> s
{1, 2, 3.5, 'psp', 12, 45, 90}
>>> s.update([11,22])
>>> s
{1, 2, 3.5, 11, 'psp', 12, 45, 22, 90}
>>> s1={11,22,33,44}
>>> s.update(s1)
>>> s
{1, 2, 3.5, 33, 11, 'psp', 12, 45, 44, 22, 90}
```

Add Any Iterable

The object in the update() method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

```
>>> set1 = {"apple", "banana", "cherry"}
>>> l1 = ["kiwi", "orange"]
>>> set1.update(l1)
>>> set1
{'apple', 'kiwi', 'orange', 'cherry', 'banana'}
```

Remove elements from a set

A particular item can be removed from set using methods, discard() and remove().
using discard() if the item does not exist in the set, it remains unchanged.
But remove() will raise an error in such condition.

```
>>> s1={1,2,3,11,22,'psp','a'}
>>> s1.remove(3)
>>>s1
{1, 2, 'psp', 11, 22, 'a'}
>>> s1.remove(21)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 21
>>> s1.discard(2)
>>> s1
{1, 'psp', 11, 22, 'a'}
>>> s1.discard(21)
```

Pop(): removes and returns any random element from the set.

```
>>> set1
{'apple', 'orange', 'cherry', 'banana'}
>>> set1.pop()
'apple'
>>> set1
{'orange', 'cherry', 'banana'}
>>> set1.pop()
'orange'
>>> set1
{'cherry', 'banana'}
>>> set1.pop()
'cherry'
>>> set1.pop()
'banana'
>>> set1
set()
>>> set1.pop()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'pop from an empty set'
```

Sets can be used to carry out mathematical set operations like union, intersection, difference and symmetric difference.
We can do this with operators or methods.

```
>>> s1={1,2,3,4,5}
>>> s2={4,5,6,7,8}
```

Union

The union() method or | operator returns a new set with all items from both sets:

```
s1={1,2,3,4,5}

s2={4,5,6,7,8}

s3=s1.union(s2)  (or) s3=s1|s2

print(s1)

print(s2)

print(s3)
```

Output:

```
s1={1,2,3,4,5}

s2={4,5,6,7,8}
```

s3={1,2,3,4,5,6,7,8}

Intersection

The intersection() method will return a *new* set, that only contains the items that are present in both sets.

```
s1={1,2,3,4,5}
s2={4,5,6,7,8}
s3=s1.intersection(s2) or s3=s1&s2
print("s1=",s1)
print("s2=",s2)
print("s3=",s3)
```

output:

```
s1={1, 2, 3, 4, 5}
```

s2={4, 5, 6, 7, 8}
s3={4, 5}

intersection_update() :

The intersection_update() method will keep only the items that are present in both sets.

```
s1={1,2,3,4,5}
s2={4,5,6,7,8}
s1.intersection_update(s2)
print("s1:",s1)
print("s2:",s2)
```

Output:
s1: {4, 5}
s2: {4, 5, 6, 7, 8}

difference():

The difference() method or – operator returns the elements that are in set1 and not in set 2

```
s1={1,2,3,4,5}

s2={4,5,6,7,8}

s3=s1.difference(s2)

s4=s1-s2

print("s1:",s1)

print("s2:",s2)

print("s3:",s3)

print("s4:",s4)
```

output:

s1: {1, 2, 3, 4, 5}

s2: {4, 5, 6, 7, 8}

s3: {1, 2, 3}

s4: {1, 2, 3}

Symmetric difference

The symmetric difference of two sets is calculated by ^ operator or symmetric_difference() method. Symmetric difference of sets, it removes that element which is present in both sets.

```
s1={1,2,3,4,5}

s2={4,5,6,7,8}

s3=s1.symmetric_difference(s2)

s4=s1^s2

print("s1:",s1)

print("s2:",s2)

print("s3:",s3)

print("s4:",s4)
```

Output:

s1: {1, 2, 3, 4, 5}

s2: {4, 5, 6, 7, 8}

s3: {1, 2, 3, 6, 7, 8}

s4: {1, 2, 3, 6, 7, 8}

Set comparisons

Python allows us to use the comparison operators i.e., <, >, == with the sets by using which we can check whether a set is a subset, superset, or equivalent to other set. The boolean true or false is returned depending upon the items present inside the sets.

```
s1={1,2,3,4,5}

s2={4,5}
```

```
s3=s1>s2 # s1 is super set of s2
s4=s2<s1 # s2 is subset of s1
print("s1:",s1)
print("s2:",s2)
print("s3:",s3)
print("s4:",s4)
s5={1,2,3,4,5}
print(s1==s5)
```